SWYNEX TECHNOLOGIES

SECURITY ASSESSMENT REPORT

DVWA Local Lab – Baseline Security Assessment

INTERNSHIP TASK 4

Prepared by	Sharwari Ghodke
Organisation	SWYNEX Technologies
Assessment date	4 November 2026
Assessment environment	Local DVWA lab on Windows 11 / XAMPP

CONFIDENTIAL – FOR INTERNSHIP SUBMISSION

Contents

1. Executive Summary
2. Scope and Objectives
3. Testing Environment
4. Assessment Methodology
5. Assessment Results and Observations
6. Findings and Risk Assessment
7. Recommendations
8. Retesting and Verification
9. Evidence Documentation
10. Limitations
11. Conclusion
12. References

1. Executive Summary

This report documents a limited baseline security assessment of Damn Vulnerable Web Application (DVWA) deployed in a local Windows 11 environment using XAMPP. The work was performed as part of Internship Task 4 for SWYNEX Technologies. The assessment focused on verifying the lab setup and observing selected application functions using ordinary, non-malicious inputs.

The DVWA security level was set to Impossible during the recorded checks. Normal-input observations were made for SQL Injection and Reflected Cross-Site Scripting (XSS). The File Upload and CSRF interfaces were inspected, but no file was uploaded and no password-change request was submitted. No exploit payloads or destructive tests were performed.

No confirmed security vulnerabilities were established within this limited scope. This does not demonstrate that the application is secure; it means that the checks performed were insufficient to confirm or rule out vulnerabilities. The PHP GD extension was not installed, which was recorded as an environment limitation.

2. Scope and Objectives

2.1 Scope

- Local DVWA instance hosted at http://localhost/DVWA.
- Verification of the local XAMPP, Apache, database and PHP environment.
- Baseline observation of selected DVWA modules: SQL Injection, Reflected XSS, File Upload and CSRF.
- Recording configuration observations and available screenshot evidence.

2.2 Objectives

- Confirm that the intentionally vulnerable training application is accessible in the local lab.
- Document selected module behaviour using ordinary input and interface inspection.
- Identify configuration limitations and recommend standard defensive controls.

2.3 Out of Scope

- Testing public websites, third-party systems or networks.
- Exploit development, destructive testing, password attacks or unauthorised access.
- Comprehensive penetration testing, source-code review and formal compliance assessment.

3. Testing Environment

Component	Observed configuration
Operating system	Windows 11
Local stack	XAMPP
Web server	Apache (local instance)
PHP version	8.3.36
Database service	MySQL-compatible database service through XAMPP
Application	Damn Vulnerable Web Application (DVWA)
Application URL	http://localhost/DVWA
Database	dvwa

DVWA security level	Impossible
PHP extensions observed	mysqli and pdo_mysql available; GD not installed
PHP configuration note	allow_url_include disabled

4. Assessment Methodology

The assessment followed a cautious, observation-based process:

- Set up the local XAMPP services and verify that the database administration interface is accessible.
- Place the DVWA application in the local web root, configure its database connection and initialise its database.
- Access DVWA locally and confirm that the application dashboard and selected modules are available.
- Set the DVWA security level to Impossible for the recorded baseline observations.
- Use ordinary input in selected modules and record the visible application response.
- Inspect the File Upload and CSRF forms without submitting a file or changing a password.
- Capture screenshots of relevant setup, configuration and baseline observations.

5. Assessment Results and Observations

5.1 SQL Injection – Normal Input

A normal User ID value of 1 was entered. The application displayed the record with ID 1 and the name values “admin” and “admin”. This was a baseline functionality check only. No SQL injection payload was used, and the result does not establish whether the module is vulnerable or secure.

5.2 Reflected XSS – Normal Input

The ordinary text “hello” was entered and the page displayed “hello hello”. No script or HTML payload was submitted. This confirms only the observed response to normal text; it is not an XSS vulnerability test.

5.3 File Upload Interface

The File Upload form was visible. No file was uploaded, so file type validation, content inspection, storage location and execution controls were not assessed. The environment displayed a warning that the PHP GD module was not installed. This is recorded as a configuration limitation and not as a confirmed application vulnerability.

5.4 CSRF Interface

The password-change form displayed fields for the current password, new password and confirmation. No password-change request was submitted. The presence or effectiveness of CSRF protection, including any token validation, was not verified. No conclusion about CSRF vulnerability can therefore be made.

6. Findings and Risk Assessment

The table distinguishes confirmed environment observations from areas that were only partially inspected. “Not rated” means there is insufficient evidence to assign a vulnerability severity.

ID	Observation	Status / Rating	Notes
F-01	PHP GD extension unavailable	Informational	Confirmed environment limitation; some image-related functionality may be affected.
F-02	SQL Injection module checked with normal input only	Not rated	No exploit test performed; security status remains undetermined.
F-03	Reflected XSS module checked with normal input only	Not rated	No script payload or output-encoding test performed.
F-04	File Upload interface inspected; no upload performed	Not rated	Upload validation and storage controls remain unassessed.
F-05	CSRF password-change form inspected; no request submitted	Not rated	Token presence and server-side validation remain unverified.

Overall assessment: No confirmed security vulnerabilities were established by the checks documented in this report. This is a scope-limited result, not a clean bill of health.

7. Recommendations

- Install and enable the PHP GD extension only if it is required for the intended DVWA lab functionality, then verify the application configuration.
- Use parameterised queries or prepared statements for database operations and apply least-privilege database permissions.
- Apply context-appropriate output encoding to untrusted data before rendering it in web pages.
- For file uploads, enforce an allowlist of file types, validate content, generate safe server-side filenames, impose size limits and store uploads outside executable web directories.
- Use unpredictable, session-bound CSRF tokens and validate them on the server for state-changing requests.
- Keep the local training environment isolated and do not expose DVWA to public networks.
- Keep the operating system, web server, PHP runtime and database components updated.

These are general defensive recommendations. They were not implemented or verified as part of this assessment.

8. Retesting and Verification

No remediation activities were carried out during this assessment, and no retesting was performed. Any future remediation should be followed by controlled verification in the authorised local lab, with before-and-after evidence recorded.

9. Evidence Documentation

Screenshots were captured during the lab work to document the environment and selected observations. The evidence folder should be submitted alongside this report. The captured evidence categories include:

- XAMPP service status and local environment.
- DVWA setup, login, dashboard and security-level configuration.
- Available DVWA security modules and application information.
- PHP configuration and the GD extension warning.
- SQL Injection normal-input result and Reflected XSS normal-input result.
- File Upload interface and CSRF password-change form.
- DVWA database and table presence in the database administration interface.

Evidence note: The screenshots are maintained separately from this document. Ensure that the submitted evidence folder contains the actual screenshots corresponding to the observations above; this report does not embed or fabricate screenshot images.

10. Limitations

- The assessment was limited to a local DVWA training environment.
- The recorded checks were performed with the DVWA security level set to Impossible.
- Only selected modules were reviewed; other DVWA modules were not assessed.
- SQL Injection and Reflected XSS checks used ordinary input only, not vulnerability payloads.
- The File Upload workflow was not tested by uploading a file.
- The CSRF workflow was not submitted, and token validation was not verified.
- No source-code review, comprehensive penetration test or independent security audit was performed.
- The PHP GD extension was unavailable during the recorded environment check.

11. Conclusion

The local DVWA environment was configured and accessed successfully, and selected application interfaces were reviewed using safe baseline checks. The assessment documented normal-input behaviour, the visible File Upload and CSRF forms, and the missing PHP GD extension. No confirmed vulnerabilities were established within the limited scope. Further authorised testing would be required to evaluate the security of the selected modules and the wider application. The observations and limitations in this report should be considered when interpreting the results.

12. References

- Damn Vulnerable Web Application (DVWA), official project repository: https://github.com/digininja/DVWA
- Apache Friends, XAMPP: https://www.apachefriends.org/
- PHP Documentation: https://www.php.net/docs.php
- OWASP Foundation: https://owasp.org/
- OWASP Cheat Sheet Series: https://cheatsheetseries.owasp.org/